

Paimon API 模块核心架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块：paimon-api

生成工具：AI Architecture Analyzer

生成日期：2026年03月28日

第1章 模块概述

1.1 模块定位

paimon-api 是 Apache Paimon 项目的轻量级公共 API 模块，为上层应用（Flink、Spark、Hive 等引擎）和 REST Catalog 客户端提供统一的编程接口。该模块刻意避免引入 Hadoop 等重型依赖，确保独立、轻量的部署特性。

模块共包含 205 个 Java 源文件，分布在 23 个包中，最大源文件为 CoreOptions.java（约 195KB），代码总量约 2.5 万行。rest 包及其子包（REST Catalog 客户端）占全部文件的 48%（99 个文件），是模块的核心功能域。

1.2 核心目标

- 统一类型系统：提供完整的 DataType 类型体系（22+ 种数据类型），基于 Visitor 模式实现类型安全的操作分派
- 表/视图/函数 API 定义：定义 Schema、SchemaChange、View、ViewChange、Function、FunctionDefinition 等核心数据模型
- REST Catalog 客户端：提供完整的 REST API 客户端实现，包括 HTTP 通信、认证、错误处理、重试、拦截器等
- 配置选项框架：类型安全的 ConfigOption/Options 配置体系，支持动态配置和优雅降级
- SPI 工厂机制：基于 Java ServiceLoader 的可插拔组件发现与加载框架
- 轻量无依赖：不依赖 Hadoop/HDFS 等重型框架，可独立嵌入任何 Java 应用

1.3 包结构概览

包名	文件数	功能描述
o.a.p.rest	99	REST Catalog 客户端（HTTP、Auth、Models、Interceptors）
o.a.p.types	35	数据类型体系（DataType + 22 种具体类型 + Visitor）
o.a.p.options	12	配置选项框架（ConfigOption、Options、CoreOptions）
o.a.p.schema	8	Schema 管理（Schema、TableSchema、SchemaChange）
o.a.p.factories	5	SPI 工厂机制（Factory、FactoryUtil）
o.a.p.view	4	视图 API 定义（View、ViewSchema、ViewChange）
o.a.p.function	5	函数 API（Function、FunctionDefinition、FunctionChange）
o.a.p.catalog	8	Catalog 抽象层（CatalogContext、CatalogFactory）
o.a.p.data	12	数据行 API（InternalRow、BinaryRow 等）

包名	文件数	功能描述
o.a.p.snapshot	5	快照数据模型 (Snapshot 、CommitKind)
o.a.p.utils	12	工具类 (Preconditions 、StringUtils 等)

第2章 核心架构设计

2.1 整体架构分层

paimon-api 采用清晰的分层架构设计，自顶向下分为四个核心层次：API 接口层、REST 客户端层、数据模型层和基础设施层。各层之间通过接口解耦，上层依赖下层的抽象而非具体实现。

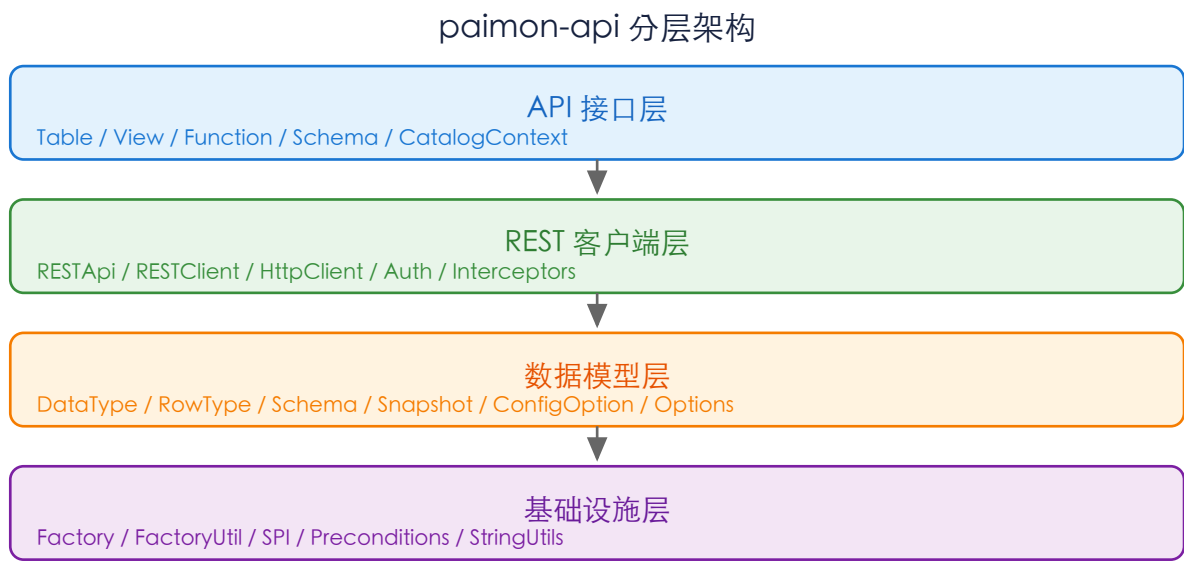


图2-1 paimon-api 整体分层架构图

2.2 REST 客户端架构

REST 客户端是 paimon-api 最核心的功能域，占全部代码量的 48%。其架构以 RESTApi 为 Facade 入口，通过 HttpClient 执行 HTTP 请求，支持多种认证方式（Bearer Token、阿里云 DLF），并提供错误处理、重试和拦截器扩展机制。

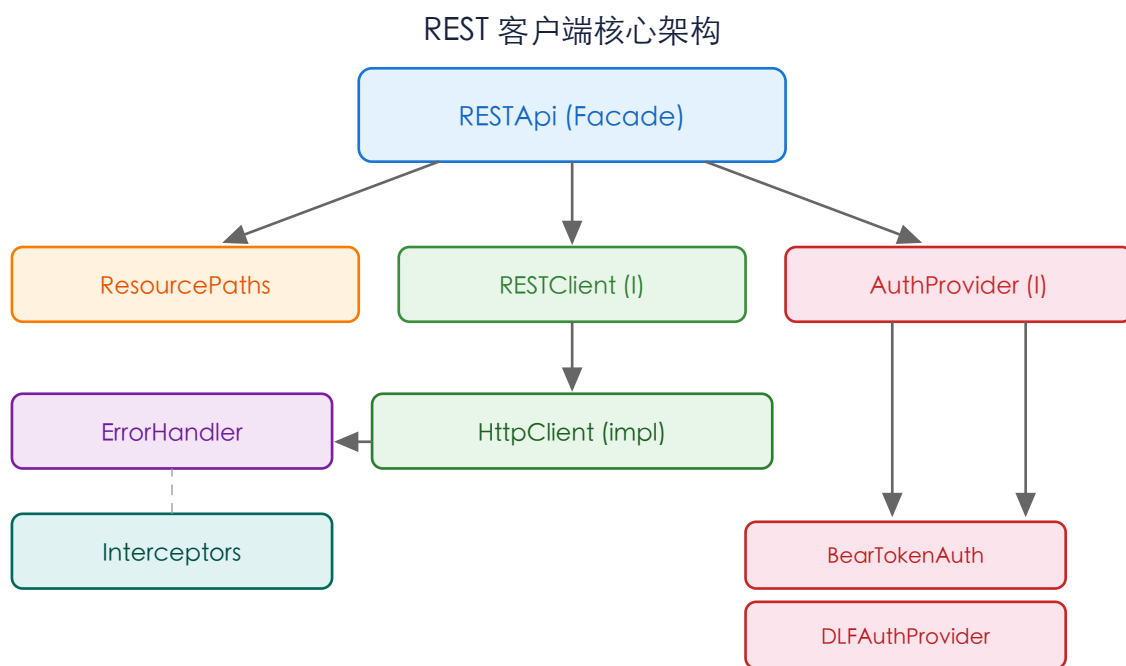


图2-2 REST 客户端核心架构图

2.3 数据类型体系

paimon-api 定义了完整的数据类型体系，以 `DataType` 为抽象基类，通过 `DataTypeVisitor` 接口实现 Visitor 模式。类型体系覆盖了数值类型、字符类型、时间类型、复合类型等 22 种以上数据类型，为跨引擎数据交换提供统一的类型语义。

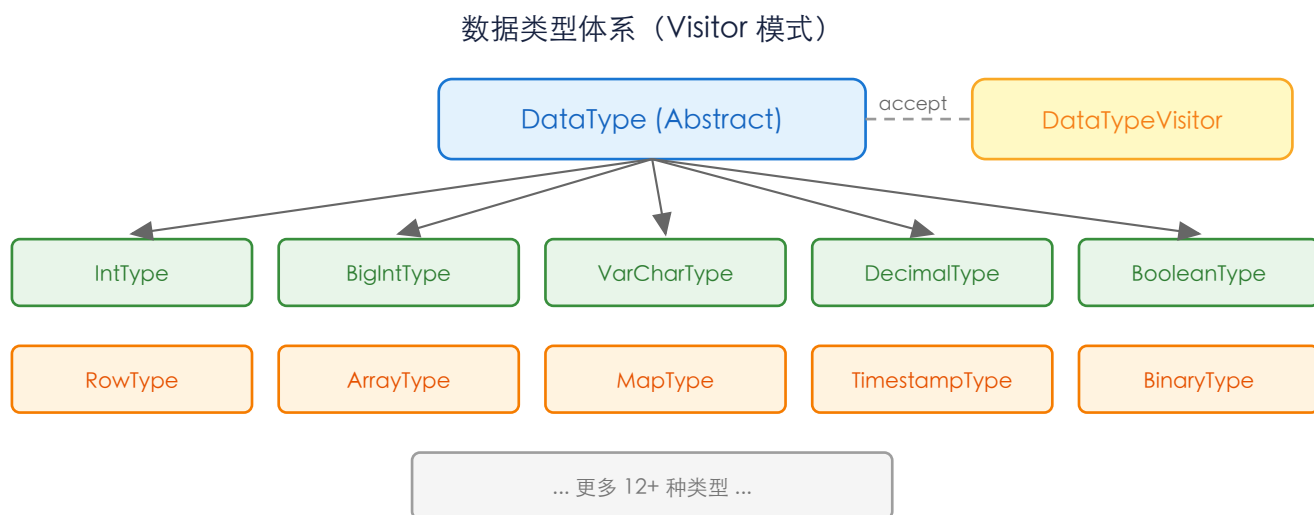


图2-3 数据类型体系 (Visitor 模式)

2.4 配置选项框架

配置选项框架由 `ConfigOption<T>`、`Options` 和 `ConfigOptions` 三部分组成。`ConfigOption` 提供类型安全的配置定义（支持泛型参数类型、默认值、fallback keys），`Options`

作为线程安全的配置存储（内部使用 `HashMap<String, String>`），`ConfigOptions` 提供流畅的 Builder API 用于构建配置选项。`CoreOptions` 汇聚了 Paimon 全部核心配置项定义。

2.5 Schema 管理架构

Schema 管理涉及三个核心类：`Schema`（用户定义的表结构，包含 `fields/partitionKeys/primaryKeys/options/comment`）、`TableSchema`（包含 `schemald/highestFieldId/version/timeMillis` 的完整运行时 Schema）和 `SchemaChange`（定义 11 种变更操作的多态接口，使用 `Command` 模式）。`Schema` 与 `TableSchema` 之间通过 `Schema.fromTableSchema()` 和 `TableSchema` 构造转换。

第3章 核心类设计模式详解

3.1 DataType / DataTypeVisitor — Visitor 模式

`DataType` 是整个类型系统的抽象基类，定义了 `accept(DataTypeVisitor<R>)` 方法。`DataTypeVisitor<R>` 为所有 22+ 种具体类型定义了 `visit` 方法。每个具体类型（如 `IntType`、`VarCharType`、`RowType`）在 `accept()` 中回调对应的 `visit` 方法，实现了「双重分派」。

设计价值：新增类型操作（如序列化、打印、转换）只需实现 `DataTypeVisitor` 接口，无需修改任何已有类型类。这遵循了开闭原则（OCP），在 Paimon 跨引擎场景中，不同引擎可以实现自己的 `Visitor` 来处理类型映射。

关键方法：

- `DataType.accept(DataTypeVisitor<R>)` — 接受访问者，触发双重分派
- `DataType.copy(boolean isNullable)` — 不可变复制，保持类型不变性
- `DataTypeVisitor.visit(XxxType)` — 22+ 个 `visit` 方法对应各具体类型

```
// Visitor ■■■■■■
public abstract class DataType {
    public abstract <R> R accept(DataTypeVisitor<R> visitor);
}

public class IntType extends DataType {
    @Override
    public <R> R accept(DataTypeVisitor<R> visitor) {
        return visitor.visit(this);
    }
}
```

3.2 Schema.Builder / RowType.Builder — Builder 模式

`Schema` 和 `RowType` 都采用 `Builder` 模式进行构建。`Schema.Builder` 提供 `column()`、`primaryKey()`、`partitionKeys()`、`option()` 等流畅方法，支持链式调用。`Builder` 在 `build()` 时执行验证逻辑（如将主键字段自动规范化为 `NOT NULL`），确保构建出的对象满足业务约束。

关键方法：

- `Schema.newBuilder()` — 创建新的 `Builder` 实例
- `Builder.column(name, type)` — 添加列定义
- `Builder.primaryKey(fields...)` — 设置主键
- `Builder.build()` — 执行验证并构建 `Schema` 对象

```
// Builder ■■■■■ Schema
Schema schema = Schema.newBuilder()
    .column("id", DataTypes.INT().notNull())
    .column("name", DataTypes.VARCHAR(100))
```

```
.primaryKey("id")
.partitionKeys("dt")
.option("bucket", "4")
.build();
```

3.3 Factory / FactoryUtil — Abstract Factory + SPI 模式

Factory 接口定义了统一的 SPI 工厂契约，通过 identifier() 方法唯一标识每个工厂实现。FactoryUtil 是工厂发现的核心工具类，使用 Java ServiceLoader 机制扫描 classpath 上的所有 Factory 实现，并利用 Caffeine 缓存加速重复查找。discoverFactory() 方法根据 identifier 精确匹配目标工厂。

关键方法：

- Factory.identifier() — 返回工厂的唯一标识符（如 "filesystem"、"rest"）
- FactoryUtil.discoverFactory(ClassLoader, Class, String) — SPI 发现并匹配工厂
- FactoryUtil.discoverIdentifiers(ClassLoader, Class) — 列出所有已注册的标识符

```
// SPI ■■■■■■
public interface Factory {
    String identifier();
}

// ■■■ ServiceLoader + Caffeine ■■■
CatalogFactory factory = FactoryUtil.discoverFactory(
    classLoader, CatalogFactory.class, "rest");
```

3.4 AuthProvider / DLFAuthProvider — Strategy 模式

认证体系采用 Strategy 模式。AuthProvider 接口定义了 mergeAuthHeader() 方法，不同认证策略（BearerAuthProvider、DLFAuthProvider）实现各自的认证头生成逻辑。DLFAuthProvider 内部还嵌套了 DLFRequestSigner 策略接口，支持 DLFDefaultSigner 和 DLFOpenApiSigner 两种签名算法。AuthProviderFactory 继承 Factory 接口，使用 SPI 发现机制动态加载认证实现。

关键方法：

- AuthProvider.mergeAuthHeader(headers, request) — 将认证信息注入请求头
- DLFAuthProvider.checkTokenExpire() — 检查并刷新过期 Token
- DLFRequestSigner.sign(request, credential) — 策略签名接口

3.5 RESTApi — Facade 模式

RESTApi 是 REST Catalog 客户端的统一入口（Facade），封装了 HttpClient、AuthProvider、ResourcePaths 等底层组件。外部调用者只需通过 RESTApi 的 listDatabases()、createTable()、getTable() 等方法即可完成 Catalog 操作，无需关心 HTTP 请求构建、认证头注入、错误处理等细节。

关键方法：

- `RESTApi.listDatabases()` — 列出所有数据库
- `RESTApi.createTable(request)` — 创建表
- `RESTApi.getTable(dbName, tableName)` — 获取表详情
- `RESTApi.alterTable(request)` — 修改表 (SchemaChange)
- `RESTApi.dropTable(dbName, tableName)` — 删除表

3.6 SchemaChange — Command 模式

SchemaChange 采用 Command 模式，定义了 11 种独立的变更命令类：`SetOption`、`RemoveOption`、`UpdateComment`、`AddColumn`、`RenameColumn`、`DropColumn`、`UpdateColumnType`、`UpdateColumnNullability`、`UpdateColumnComment`、`UpdateColumnPosition`、`MaterializedTableChange`。每种变更都是一个不可变值对象，通过 Jackson `@JsonTypeInfo/@JsonSubTypes` 实现多态 JSON 序列化。

11 种变更操作：

- `SetOption` / `RemoveOption` — 设置/删除表选项
- `AddColumn` / `DropColumn` / `RenameColumn` — 列的增删改名
- `UpdateColumnType` / `UpdateColumnNullability` — 修改列类型和可空性
- `UpdateColumnComment` / `UpdateColumnPosition` — 修改列注释和位置
- `UpdateComment` — 更新表注释
- `MaterializedTableChange` — 物化表专用变更

3.7 DefaultExceptionHandler — Singleton + Template Method 模式

`DefaultExceptionHandler` 使用 Singleton 模式提供全局唯一的错误处理器实例。`ExceptionHandler` 基类定义了 Template Method 模式的 `handle()` 方法，根据 HTTP 状态码分派到不同的处理逻辑：4xx 客户端错误映射为 `BadRequestException`/`ForbiddenException`/`NotFoundException`，5xx 服务端错误映射为 `ServiceFailureException` 等。

第4章 核心流程图

4.1 RESTApi 初始化流程

RESTApi 的初始化过程涉及配置合并、HTTP 客户端创建、认证提供者加载和资源路径构建。核心步骤包括：从 CatalogContext 中读取 REST 配置、通过 SPI 发现并创建 AuthProvider、初始化 HttpClient (Apache HttpClient 5)、构建 ResourcePaths 路径模板。

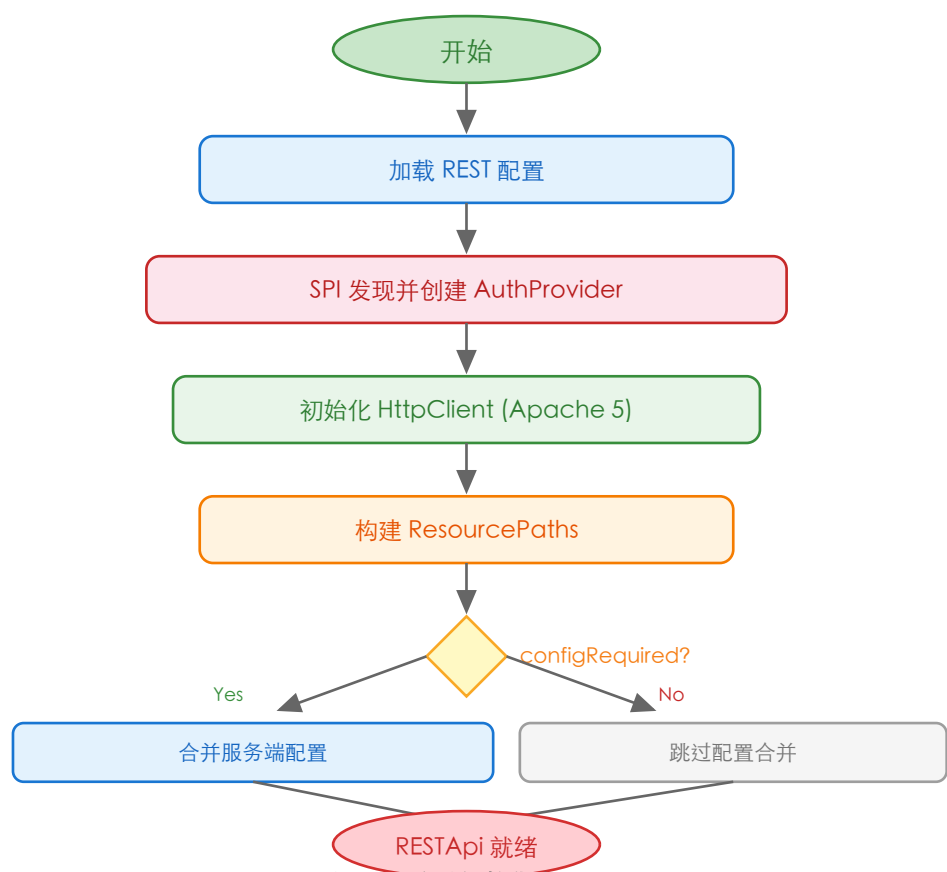


图4-1 RESTApi 初始化流程图

4.2 REST 请求执行流程

每次 REST API 调用的执行流程包括：路径构建、请求体序列化、认证头注入、HTTP 请求发送、响应处理和错误映射。

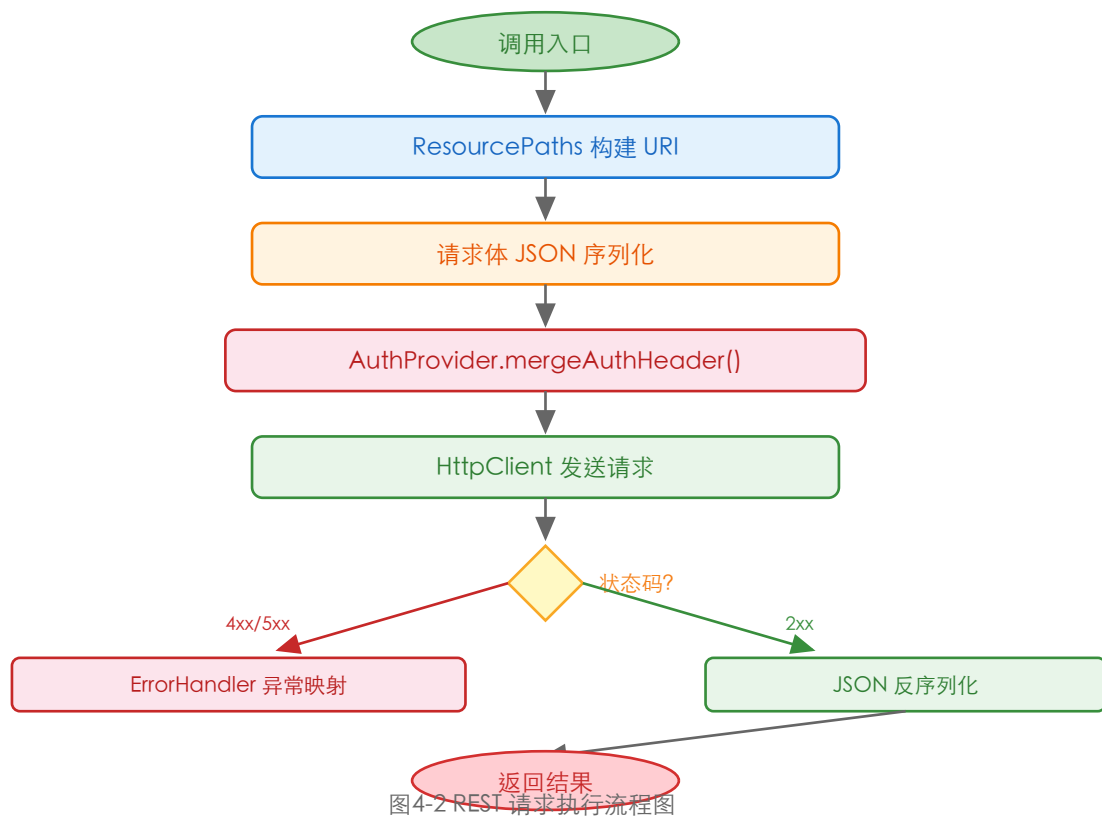


图4-2 REST 请求执行流程图

4.3 DLF 认证流程

DLF (Data Lake Formation) 认证是阿里云场景的核心认证方式。支持多种 Token 加载方式和签名算法，包含自动刷新和缓存机制。

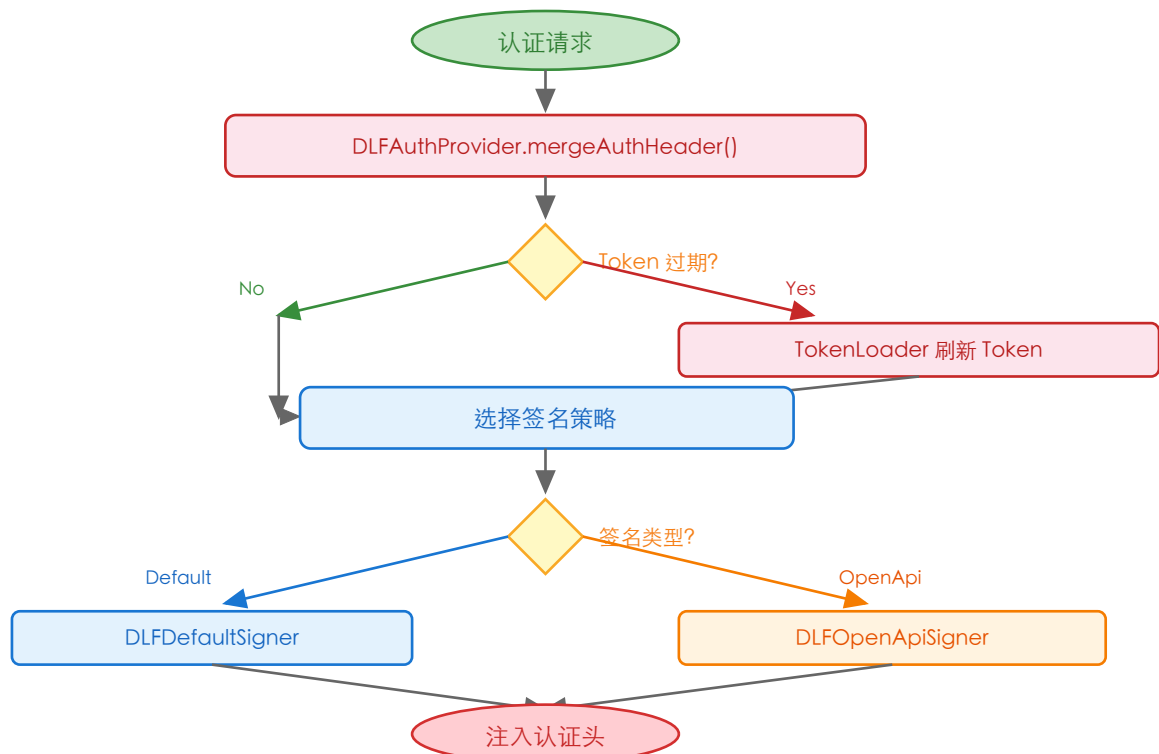


图4-3 DLF 认证流程图

第5章 核心时序图

5.1 创建表（createTable）时序图

创建表是 REST Catalog 最核心的操作之一，涉及 RESTApi、HttpClient、AuthProvider、ResourcePaths 和远程 REST 服务器之间的协作。以下时序图展示了完整的交互流程。

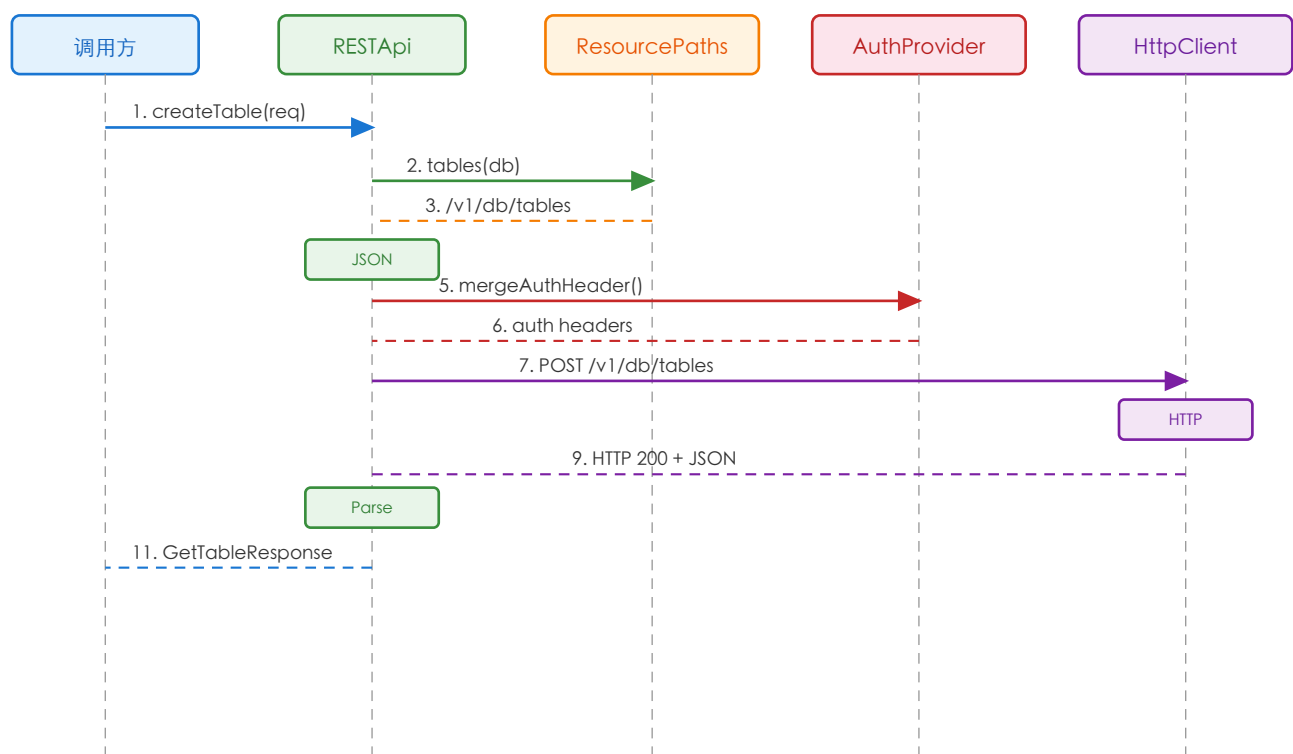


图5-1 创建表（createTable）时序图

5.2 SPI 工厂发现时序图

SPI 工厂发现通过 FactoryUtil.discoverFactory() 方法执行，涉及 ServiceLoader 扫描和 Caffeine 缓存。首次调用时通过 ServiceLoader 加载所有实现，后续调用命中缓存。

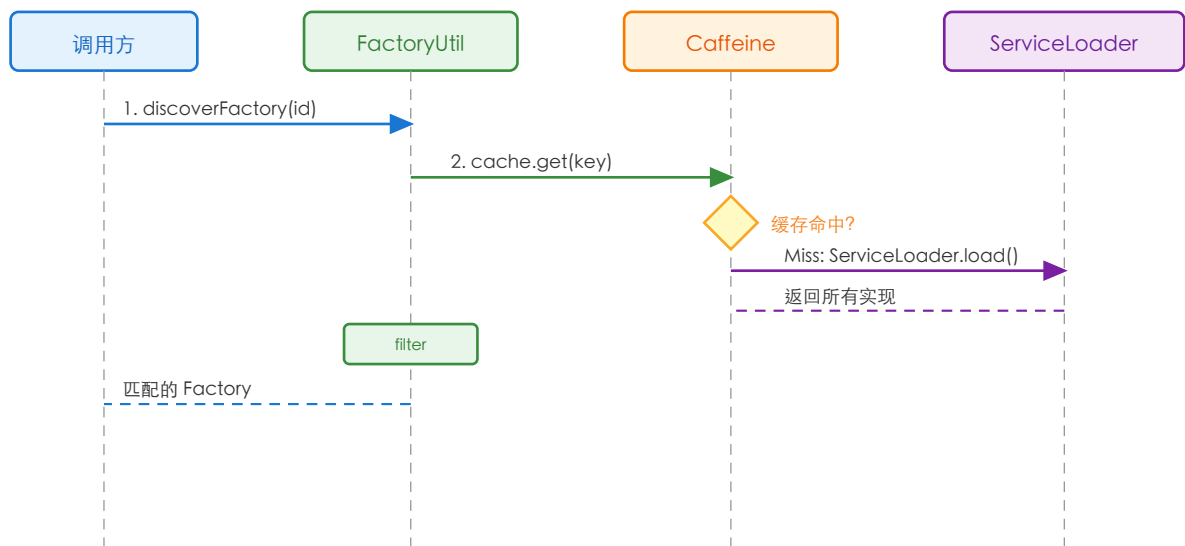


图5-2 SPI 工厂发现时序图

第6章 关键场景调用链

6.1 场景一：列出所有表（listTables）

触发条件：用户通过 REST Catalog 查询指定数据库下的所有表。

调用链路：

步骤	类.方法	说明
1	RESTApi.listTables(dbName)	Facade 入口，接收数据库名参数
2	ResourcePaths.tables(dbName)	构建 URI: /v1/{prefix}/databases/{db}/tables
3	AuthProvider.mergeAuthHeader()	注入认证头（Bearer Token 或 DLF 签名）
4	HttpClient.get(uri, headers)	发送 GET 请求到 REST 服务器
5	ErrorHandler.handle(response)	检查响应状态码，异常映射
6	JSON → ListTablesResponse	反序列化响应为 ListTablesResponse 对象

6.2 场景二：创建表（createTable）

触发条件：用户通过 REST Catalog 创建新表，传入 Schema 定义。

调用链路：

步骤	类.方法	说明
1	RESTApi.createTable(request)	Facade 入口，接收 CreateTableRequest
2	Schema.Builder.build()	验证 Schema（主键 NOT NULL 规范化等）
3	ResourcePaths.tables(dbName)	构建 URI
4	ObjectMapper.writeValueAsString()	序列化请求体为 JSON
5	AuthProvider.mergeAuthHeader()	注入认证头
6	HttpClient.post(uri, body, headers)	发送 POST 请求
7	ErrorHandler.handle(response)	错误处理（409 = AlreadyExist）
8	JSON → GetTableResponse	反序列化创建结果

6.3 场景三：修改表（alterTable）

触发条件：用户通过 REST Catalog 提交 SchemaChange 列表修改表结构。

调用链路：

步骤	类.方法	说明
1	RESTApi.alterTable(request)	Facade 入口，接收 AlterTableRequest
2	SchemaChange → JSON	多态序列化（@JsonTypeInfo 分派 11 种类型）
3	ResourcePaths.table(db, table)	构建 URI: /v1/.../tables/{table}
4	AuthProvider.mergeAuthHeader()	注入认证头
5	HttpClient.post(uri, body, headers)	发送 POST 请求
6	JSON → GetTableResponse	反序列化修改后的表信息

6.4 场景四：DLF 认证签名

触发条件：使用阿里云 DLF 认证的 REST API 请求。

调用链路：

步骤	类.方法	说明
1	DLFAuthProvider.mergeAuthHeader()	入口，检查 Token 状态
2	checkTokenExpire()	判断 Token 是否过期
3	TokenLoader.loadToken()	过期时通过 SimpleHttpClient 刷新
4	DLFRequestSigner.sign()	策略选择签名器执行签名
5	DLFDefaultSigner / DLFOpenApiSigner	生成 HMAC-SHA256 签名
6	headers.put("Authorization", sig)	将签名注入请求头

6.5 场景五：获取快照（getSnapshot）

触发条件：查询表的最新快照信息，包含 manifest 列表、记录计数和水位线。

调用链路：

步骤	类.方法	说明
1	RESTApi.getTableSnapshot(db, table)	Facade 入口
2	ResourcePaths.tableSnapshot(db, table)	构建快照查询 URI

步骤	类.方法	说明
3	AuthProvider.mergeAuthHeader()	认证头注入
4	HttpClient.get(uri, headers)	发送 GET 请求
5	JSON → Snapshot	反序列化为 Snapshot 对象
6	Snapshot.id/commitKind/watermark	返回快照元数据

第7章 配置与扩展

7.1 REST Catalog 配置项

配置项	默认值	说明
rest.uri	(必填)	REST Catalog 服务器 URI
rest.token	(空)	Bearer Token 认证令牌
rest.auth-provider	(空)	认证提供者标识 (SPI)
rest.prefix	(空)	API 路径前缀
rest.thread-pool-size	2	HTTP 连接池线程数
rest.connection-timeout	60s	HTTP 连接超时时间
rest.read-timeout	60s	HTTP 读取超时时间
rest.max-retries	3	最大重试次数
rest.page-size	100	分页查询每页大小

7.2 核心选项 (CoreOptions 精选)

配置项	默认值	说明
bucket	1	桶数量
bucket-key	(空)	桶键字段，默认使用主键
write-mode	auto	写入模式 (append-only / change-log)
merge-engine	deduplicate	合并引擎 (deduplicate / partial-update / aggregation)
changelog-producer	none	Changelog 生产模式
snapshot.time-retained	1h	快照保留时间
file.format	orc	文件格式 (orc / parquet / avro)

7.3 SPI 扩展点

paimon-api 提供了多个 SPI 扩展点，开发者可以通过实现对应接口并在 META-INF/services 中注册来扩展 Paimon 功能：

扩展点接口	SPI 文件	用途
CatalogFactory	o.a.p.factories.CatalogFactory	自定义 Catalog 实现（如 REST、Hive、JDBC）
AuthProviderFactory	o.a.p.rest.auth.AuthProviderFactory	自定义认证方式（如 OAuth2、Kerberos）
Factory	o.a.p.factories.Factory	通用工厂扩展基础接口

扩展示例：要添加新的认证方式，需要：

- 实现 AuthProviderFactory 接口，重写 identifier() 和 create() 方法
- 实现 AuthProvider 接口，重写 mergeAuthHeader() 方法
- 在 META-INF/services/o.a.p.rest.auth.AuthProviderFactory 中注册全类名
- 配置 rest.auth-provider = "your-identifier" 即可自动加载

第8章 设计亮点与总结

8.1 设计亮点

- 轻量无依赖设计：paimon-api 刻意不引入 Hadoop/HDFS 等重型依赖，使得该模块可以独立嵌入任何 Java 应用中。这在微服务和云原生场景中尤为重要，显著降低了部署包大小和启动时间。
- Visitor 模式驱动的类型系统：22+ 种数据类型通过 Visitor 模式实现双重分派，新增类型操作（如引擎类型映射）只需实现 Visitor 接口，完美遵循开闭原则。
- 多层次 SPI 扩展体系：Factory + FactoryUtil + Caffeine 缓存的组合，支持 CatalogFactory、AuthProviderFactory 等多个维度的插件化扩展，加载性能优异。
- 多策略认证框架：AuthProvider Strategy 模式支持 Bearer Token 和阿里云 DLF 两种认证方式，DLF 内部又通过 DLFRequestSigner 嵌套策略支持两种签名算法，架构灵活且可扩展。
- Schema 变更的 Command 模式：11 种 SchemaChange 命令配合 Jackson 多态序列化，支持 REST 传输和持久化，可组合、可回放、可审计。
- 完善的 RESTApi Facade：将 HTTP 通信、认证、序列化、错误处理等复杂逻辑统一封装，外部调用者只需面对简洁的高层 API。

8.2 设计限制与改进方向

- CoreOptions 体量过大：CoreOptions.java 约 195KB，汇聚了几百个配置项，可考虑按功能域拆分为 StorageOptions、CompactionOptions 等子类。
- REST 模型类数量多：rest.models 包含大量 Request/Response 类，可考虑引入代码生成（如 OpenAPI codegen）减少手写维护成本。
- 认证扩展性：目前仅支持 Bearer Token 和 DLF 两种认证方式，OAuth2、Kerberos 等通用认证可通过 SPI 扩展但需额外开发。

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★★	清晰的分层架构，包划分合理，职责边界清晰
设计模式运用	★★★★★	7+ 种设计模式（Visitor/Builder/Strategy/Facade/Command/Singleton/Abstract Factory）恰到好处地应用
可扩展性	★★★★☆	SPI 工厂机制和 Strategy 模式提供了良好的扩展点，但部分领域仍需开发新实现
API 设计	★★★★★	RESTApi Facade 封装简洁，Schema Builder 流畅易用，ConfigOption 类型安全

评估维度	评分	说明
轻量级设计	★★★★★	无重型依赖，可独立部署，适合云原生和微服务场景
文档完整度	★★★★☆☆	代码注释覆盖率中等，部分复杂逻辑缺少详细文档

8.4 总结

paimon-api 是一个设计精良的轻量级 API 模块，通过清晰的分层架构和多种设计模式的恰当运用，实现了高内聚、低耦合的模块设计。REST 客户端架构（占 48% 代码量）是核心亮点，Facade + Strategy + SPI 的组合提供了灵活的扩展能力。Visitor 模式驱动的类型系统和 Command 模式的 Schema 变更机制体现了对开闭原则的深入理解。该模块可作为 Java API 模块设计的参考范例。